

Modos de Cifrado en Bloque

Criptografía y Seguridad (72.44) — ITBA | Guía de estudio para parcial

1. Introducción: por qué se necesitan modos

Un cifrador de bloque opera sobre bloques de tamaño fijo n (p. ej. 128 bits en AES). En la práctica los mensajes suelen ser más grandes (o más pequeños) que un bloque:

- **Mensaje más pequeño que un bloque** — se aplica *padding*. Dos esquemas comunes:
 - *Simple Pad*: completar con ceros (el receptor debe conocer el largo real del mensaje).
 - *Des Pad* (ISO/IEC 9797-1 M2): agregar un bit 1 y luego ceros; puede requerir un bloque adicional completo.
- **Mensaje más grande que un bloque** — dividir en bloques $M_1, M_2, \dots, M_\ell$, aplicar padding al último si necesario y transformar cada bloque según un **modo de encadenamiento**.

Objetivo de un modo: extender una primitiva de bloque (que solo maneja n bits) a mensajes de longitud arbitraria, preservando propiedades de seguridad como CPA-Security.

Hay varios modos con propiedades distintas; elegir el incorrecto puede romper la seguridad aunque la primitiva subyacente sea segura.

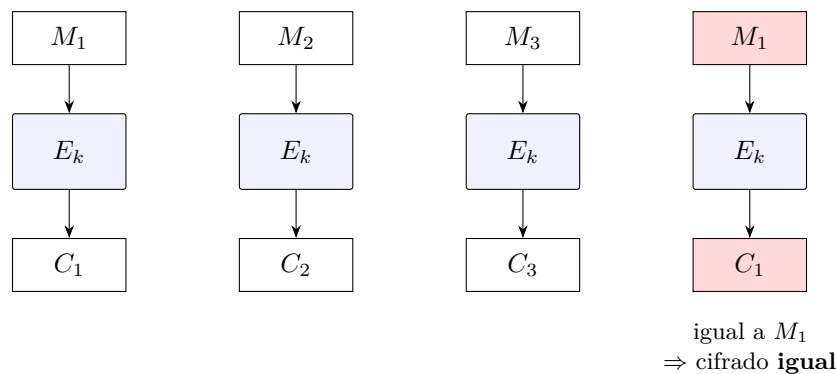
2. ECB (Electronic Codebook)

Definición ECB

Trata cada bloque de forma **independiente**:

$$C_i = E_k(M_i), \quad i = 1, \dots, \ell.$$

No usa IV ni encadenamiento. El cifrado es **determinista**.



ECB: cada bloque M_i se cifra independientemente con E_k . Dos bloques de texto plano iguales producen cifrados iguales (revelando patrones).

¡OJO!

ECB NO es CPA-Secure. No utilizar en ningún sistema real. Al ser determinista, bloques de texto plano *iguales* $\Rightarrow$ cifrados *iguales*: se revelan patrones en el mensaje. El ejemplo clásico es la imagen del pingüino de Linux cifrada con ECB, donde la silueta del animal sigue siendo visible. Esto se pregunta en recuperatorios (2025): identificar por qué ECB filtra estructura.

3. CBC (Cipher Block Chaining)

Definición CBC

Usa la salida del bloque anterior como entrada del siguiente. Requiere un **vector de inicialización** $C_0 = IV$ elegido aleatoriamente.

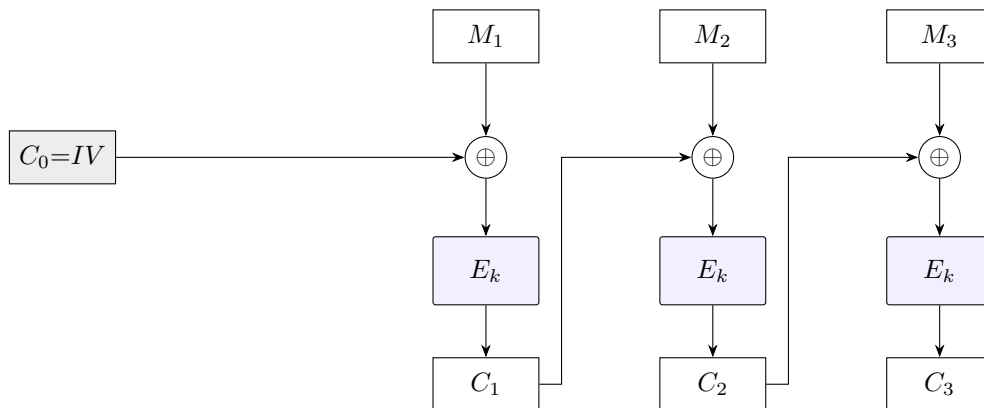
Cifrado:

$$C_i = E_k(M_i \oplus C_{i-1}), \quad C_0 = IV.$$

Descifrado: despejando M_i (aplicar D_k y deshacer el XOR):

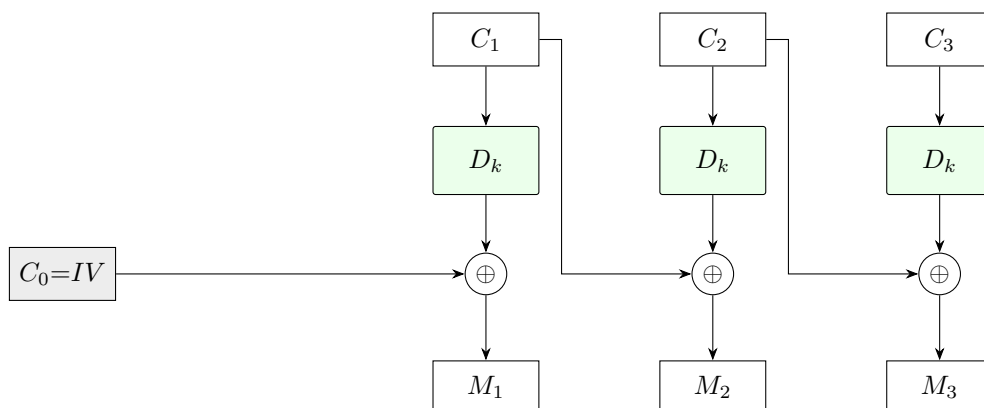
$$M_i = D_k(C_i) \oplus C_{i-1}.$$

Diagrama de cifrado CBC



CBC (cifrado): $C_i = E_k(M_i \oplus C_{i-1})$ con $C_0 = IV$. Cada bloque de texto plano se XOR-ea con el cifrado anterior antes de pasar por E_k .

Diagrama de descifrado CBC



CBC (descifrado): $M_i = D_k(C_i) \oplus C_{i-1}$. El cifrado pasa por D_k y luego se deshace el XOR con C_{i-1} .

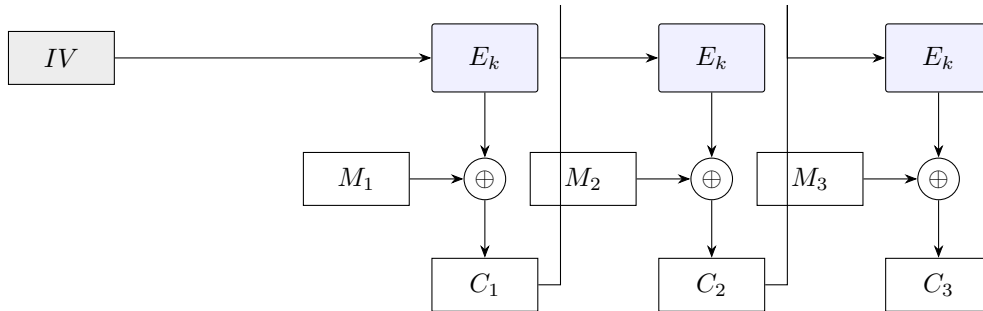
CBC con una primitiva pseudoaleatoria (PRP) e IV **aleatorio** (no predecible) es **CPA-Secure**. Si el IV es predecible, la seguridad se rompe (ataque BEAST en TLS 1.0).

4. CFB (Cipher Feedback)

Definición CFB

Convierte el cifrador de bloque en un cifrador de flujo, usando **solo** E_k (no requiere D_k). Opera realimentando el *cifrado* anterior:

$$C_i = E_k(C_{i-1}) \oplus M_i, \quad C_0 = IV.$$



CFB: usa solo E_k . Se cifra el bloque anterior (IV el primero, luego C_{i-1}) y la salida se XOR-ea con M_i .

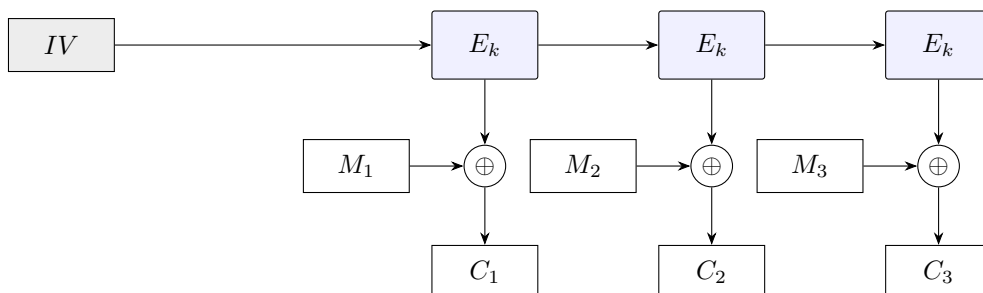
CFB es **CPA-Secure** con IV aleatorio. Útil en dispositivos embebidos donde no se puede implementar D_k .

5. OFB (Output Feedback)

Definición OFB

La *salida* de E_k (keystream) se realimenta a la entrada del bloque siguiente, **sin depender del texto**:

$$S_i = E_k(S_{i-1}), \quad S_0 = IV; \quad C_i = S_i \oplus M_i.$$



OFB: la salida de E_k (el keystream S_i) se realimenta a la entrada del bloque siguiente. A diferencia de CFB, no es el cifrado C_i sino el keystream S_i lo que se retroalimenta.

Clave: el keystream $S_1, S_2, \dots$ puede **precalcularse** antes de conocer el texto plano (no depende de M_i). OFB es **CPA-Secure** con IV aleatorio.

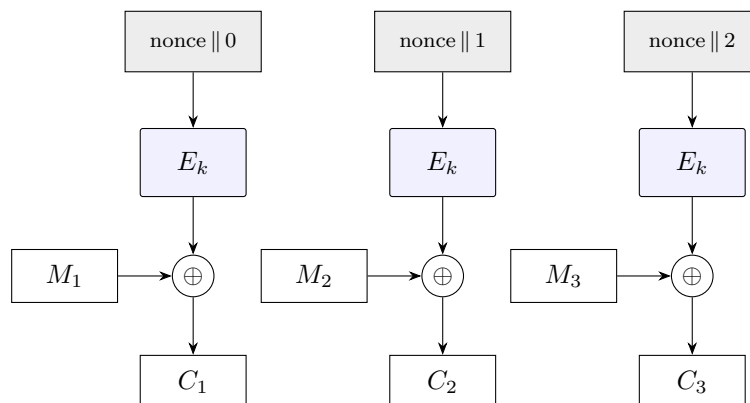
6. CTR (Counter Mode)

Definición CTR

Genera el keystream cifrando un **contador** (nonce concatenado con un índice creciente):

$$C_i = E_k(\text{nonce} \| i) \oplus M_i.$$

El descifrado usa exactamente la misma operación: $M_i = E_k(\text{nonce} \| i) \oplus C_i$.



CTR: cada bloque es independiente. El keystream se genera cifrando $\text{nonce} \| i$; luego se XOR-ea con M_i . Totalmente paralelizable y con acceso aleatorio. Nunca usa D_k .

[Examen]

CTR nunca usa D_k (pregunta 2023). Como cifrado y descifrado aplican la misma operación ($E_k(\text{nonce} \| i) \oplus \cdot$), CTR es válido incluso si la primitiva **no es invertible**. Lo mismo aplica a OFB y CFB. Este es un punto fino que suele preguntarse.

CTR es **CPA-Secure** siempre que no se repita el par (k, nonce) . Si se reutiliza, XOR-ear dos cifrados revela el XOR de los textos planos.

7. Tabla resumen de modos

Modo	CPA-Secure	Usa D_k	Paralelizable	Notas clave
ECB	No	Sí	Sí (enc+dec)	Determinista; revela patrones; nunca usar
CBC	Sí (IV aleat.)	Sí (dec)	Solo descifrado	Encadena; IV predecible $\Rightarrow$ inseguro
CFB	Sí (IV aleat.)	No	No (enc); sí (dec)	Solo E_k ; flujo a partir de bloque
OFB	Sí (IV aleat.)	No	Sí (keystream)	Keystream precalculable; no depende del texto
CTR	Sí (nonce único)	No	Sí (enc+dec)	Acceso aleatorio; nunca D_k ; muy eficiente

8. Propagación de errores en CBC (núcleo de examen)

[Examen]

Metodología general (aparece casi todos los años: 2015 PCBC, 2016 FSM, 2018/2023 CBC, 2025-1, 2025-2):

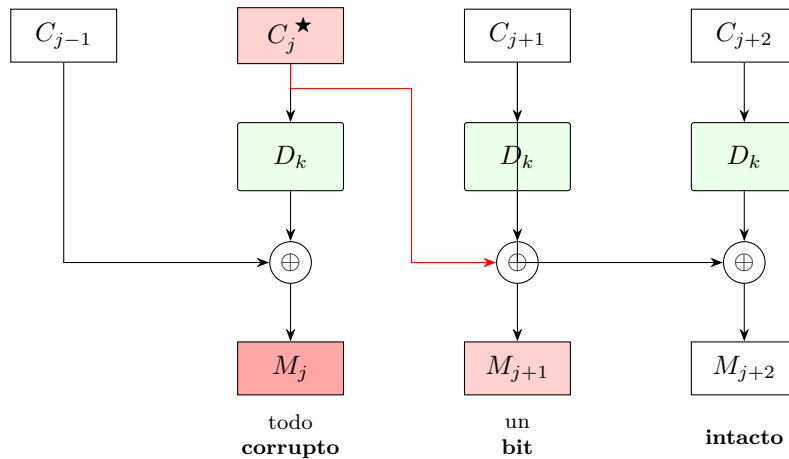
1. **Escribir la recurrencia de cifrado** del esquema dado y **despejar** M_i : aplicar D_k e invertir los XOR.
2. **Analizar un bit de error en C_j** : determinar qué bloques descifrados quedan corruptos y en qué medida.
3. **Analizar borrado/inserción de bloques**: ver qué se desincroniza.

8.1. CBC estándar: un bit de error en C_j

La ecuación de descifrado es $M_i = D_k(C_i) \oplus C_{i-1}$. Un bit erróneo en C_j aparece en dos ecuaciones:

- $M_j = D_k(C_j) \oplus C_{j-1}$: el C_j corrupto **pasa por D_k** $\Rightarrow$ corrompe **todo el bloque M_j** (un bit de entrada de un bloque produce salida completamente distinta por el efecto avalancha).
- $M_{j+1} = D_k(C_{j+1}) \oplus C_j$: aquí C_j entra por **XOR directo** $\Rightarrow$ corrompe **solo el bit correspondiente en M_{j+1}** .
- M_{j+2} y siguientes: C_j ya no aparece $\Rightarrow$ **intactos**.

En CBC estándar el error es **autolimitado a 2 bloques**: bloque M_j corrupto entero + un bit de M_{j+1} .



*Propagación de error en CBC: un bit erróneo en C_j (★) pasa por D_k y corrompe **todo** M_j ; además entra por XOR en M_{j+1} corrompiendo **solo ese bit**. Desde M_{j+2} todo queda intacto.*

8.2. PCBC (Propagating CBC) — parcial 2015

En PCBC el encadenamiento incluye también el texto plano anterior:

$$C_i = E_k(M_i \oplus C_{i-1} \oplus M_{i-1}).$$

Despejando:

$$M_i = D_k(C_i) \oplus C_{i-1} \oplus M_{i-1}.$$

Un error en C_j afecta M_j (pasa por D_k); el M_j corrupto entra como M_{i-1} en la ecuación de M_{j+1} , que a su vez contamina M_{j+2} , y así sucesivamente. El error **se propaga a todos los bloques siguientes** (nunca se autolimita).

¡OJO!

PCBC es la variante *no autolimitada*: un único bit de error en cualquier C_j destruye el descifrado desde j en adelante. Contraste con CBC estándar (solo 2 bloques afectados).

8.3. Borrado e inserción de bloques en CBC

- **Borrar** $C_0 = IV$: como $M_1 = D_k(C_1) \oplus C_0$, se pierde la referencia para M_1 (desincronización); los demás bloques se descifran con un corrimiento de índice.
- **Borrar el último bloque** C_ℓ : se pierde **solo** M_ℓ ; todos los bloques anteriores quedan intactos.
- **Insertar un bloque falso**: el bloque falso y el siguiente quedan corruptos; desde ahí el descifrado se recupera.

¡OJO!

El error más común en estos ejercicios es no distinguir el camino del bloque corrupto: **pasar por D_k corrompe el bloque entero** (efecto avalancha); **entrar por XOR corrompe solo ese bit**. Siempre contabilizar en cuantos bloques aparece C_j en las ecuaciones de descifrado.

9. Memory hooks: mnemónicos para el parcial

Modo	Regla mnemónica
ECB	E stupido — C opia tu B loque: igual entrada = igual salida.
CBC	C adena de B loques C ompartidos: XOR con el anterior.
CFB	C ifrado F eedback: el C ifrado alimenta al siguiente.
OFB	O utput: el keystream se O btiene antes del texto (precalculable).
CTR	C on T ado R : nonce + contador, todo paralelo, nunca D_k .

- “Solo E_k ”: CFB, OFB, CTR. (CBC y ECB necesitan D_k para descifrar.)
- **Paralelizable en cifrado**: ECB, CTR. (CBC, CFB, OFB no, por dependencia secuencial.)
- **Keystream independiente del texto**: OFB y CTR. (CFB y CBC dependen del texto cifrado anterior.)
- **Error autolimitado en CBC**: 2 bloques (el j entero + 1 bit del $j + 1$). PCBC: ilimitado.

10. Ejercicios tipo examen con respuesta

[Examen]

Ejercicio 1 — V/F/Justificar (estilo 2015 PCBC).

“En PCBC, si se produce un error de un bit en C_3 , entonces M_4 queda correctamente descifrado.”

Respuesta: FALSO. La ecuación de descifrado PCBC es $M_i = D_k(C_i) \oplus C_{i-1} \oplus M_{i-1}$. Un error en C_3 corrompe M_3 (pasa por D_k , efecto avalancha). El M_3 corrupto entra como M_{i-1} en la ecuación de M_4 , corrompiendo M_4 también. El error se propaga indefinidamente: $M_3, M_4, M_5, \dots$ quedan todos corruptos.

[Examen]

Ejercicio 2 — V/F/Justificar (estilo 2023 CBC con error).

“En CBC estándar, si hay un error de un bit en C_5 , entonces M_7 se descifra correctamente.”

Respuesta: VERDADERO. La ecuación es $M_i = D_k(C_i) \oplus C_{i-1}$. El bloque corrupto C_5 aparece

como:

- Entrada a D_k en M_5 : todo M_5 corrupto.
- XOR directo en M_6 : solo el bit correspondiente de M_6 corrupto.
- C_5 **no aparece** en la ecuación de $M_7 = D_k(C_7) \oplus C_6$: M_7 intacto.

El error se autolimita a 2 bloques: M_5 (entero) y M_6 (1 bit).

[Examen]

Ejercicio 3 — V/F/Justificar (estilo 2025 CTR).

“Para descifrar con CTR se necesita implementar la función de descifrado D_k del cifrador de bloque subyacente.”

Respuesta: FALSO. En CTR el descifrado es $M_i = E_k(\text{nonce}||i) \oplus C_i$, idéntico al cifrado. **Solo se usa E_k .** Esto significa que CTR (al igual que OFB y CFB) es válido incluso si la primitiva de bloque subyacente no es invertible. No se requiere ni implementa D_k en ninguno de los modos de flujo (CFB, OFB, CTR).

[Examen]

Ejercicio 4 — Pregunta abierta (propagación en CBC).

En un mensaje de 5 bloques cifrado con CBC, el tercer bloque de cifrado (C_3) se corrompe durante la transmisión. **Indicar cuáles bloques de texto plano se descifran incorrectamente y por qué.**

Respuesta. Usando $M_i = D_k(C_i) \oplus C_{i-1}$:

- $M_3 = D_k(C_3) \oplus C_2$: C_3 corrupto pasa por $D_k \Rightarrow M_3$ **entero corrupto**.
- $M_4 = D_k(C_4) \oplus C_3$: C_3 corrupto entra por XOR $\Rightarrow$ **un bit de M_4 corrupto**.
- M_1, M_2, M_5 : **intactos**.

Conclusión: bloques afectados son M_3 (completo) y M_4 (1 bit).